

49. Glisser le long d'un mur : un axe à la fois

Tester la largeur, puis la hauteur, séparément : le héros bloqué d'un côté continue de l'autre.

1. Ce qu'il faut comprendre

Maintenant le héros va partout, et en diagonale. Il occupe un carré de pixels qui peut chevaucher **deux lignes** de cases, ou **deux colonnes**. Avant d'avancer, on regarde donc **deux coins** : ceux qui vont entrer les premiers dans la case suivante.

Pour ne pas écrire quatre fois `lire(p / 8, q / 8) != MUR`, on en fait une fonction, `libre(px, py)` : « le pixel (px, py) est-il hors d'un mur ? ». Le chapitre 5 dans toute sa force : un nom clair, et chaque test tient en une ligne.

À droite, les deux coins qui entrent sont **en haut à droite** et **en bas à droite** du prochain pas : `(x + 8, y)` et `(x + 8, y + 7)`. À gauche, `(x - 1, y)` et `(x - 1, y + 7)`. En bas, `(x, y + 8)` et `(x + 7, y + 8)`. En haut, `(x, y - 1)` et `(x + 7, y - 1)`.

Ce qui est nouveau ici : un axe à la fois. On essaie d'abord le pas en largeur, **puis** le pas en hauteur, chacun avec son test. Pousse DROITE et BAS contre le mur vertical : le pas à droite est refusé, mais le pas vers le bas est accepté. Le héros **glisse** le long du mur au lieu de s'y coller.

Si l'on testait la diagonale d'un coup (« le coin en bas à droite, en `(x + 8, y + 8)`, est-il libre ? »), le moindre mur arrêterait **les deux** mouvements : le héros resterait collé dès qu'il frôle quelque chose. C'est la sensation « poisseuse » des jeux mal réglés.

À toi : retire le test des deux coins à droite pour n'en garder qu'un (`libre(x + 8, y)`), et regarde le héros entrer à moitié dans le mur par le bas.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **875 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	65 o	← la console, à l'allumage – la mise en route de la console
libre()	37 o	← main() – ta fonction
main()	337 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← ecran() – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← lire(), poser() – trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	76 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	176 o	← CopierTuiles – 9 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins
"DROITE ET BAS"	13 o	← texte() – un texte gravé

Lettres gravées : ABDEIORST

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```
lire()   × 1   27 o sur place, dans libre() + AdresseCase (10 o)
poser()  × 2   25 o sur place, dans main() + AdresseCase (10 o)
sprite() × 1   23 o sur place, dans main()
ecran()  × 2   21 o sur place, dans main() + AttendreVBlank (16 o)
bouton() × 4   20 o sur place, dans main() + LireManette (37 o)
texte()  × 1   11 o sur place, dans main() + EcrireTexte (10 o)
image()  × 1    6 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
libre() × 8   112 o sur place, dans main()
```

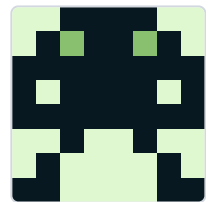
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un héros se déplace dans les 4 directions, et GLISSE le long
// des murs : bloqué d'un côté, il peut encore avancer de l'autre.
//
// Ce qui est nouveau : tester un axe à la fois.
// 1. d'abord la largeur (gauche / droite),
// 2. puis la hauteur (haut / bas), à part.
// Et pour chaque direction, on teste les DEUX coins du côté où l'on va
// (le héros fait 8 pixels : un seul coin ne suffit pas).
```

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <lire>     // lit la tuile posée sur une case
#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <poser>    // pose une tuile sur une case du fond
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};
```



à l'écran

3. UN DESSIN — MUR

```
Tuile MUR = {
    "33333333", "31111113", "31222213", "31222213",
    "31222213", "31222213", "31111113", "33333333",
};
```



à l'écran

4. UNE VARIABLE — X

```
uint8_t x = 40;           // Le héros, en pixels (coin en haut à gauche).
```

5. UNE VARIABLE — Y

```
uint8_t y = 40;

// 1 si le PIXEL (px, py) n'est pas dans un mur.
// Exemple : libre(100, 40) → lire(12, 5) : la colonne 12 est un mur → rend 0.
```

6. UNE FONCTION — LIBRE()

```
uint8_t libre(uint8_t px, uint8_t py) {
    return lire(px / 8, py / 8) != MUR;
}
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le jeu commence ici.
    ecran(0); // Dessin écran éteint.
    texte(1, 1, "DROITE ET BAS");
    for (uint8_t l = 3; l < 17; l++) poser(12, l, MUR); // un mur debout, colonne 12
    for (uint8_t c = 0; c < 20; c++) poser(c, 16, MUR); // le sol, ligne 16
    ecran(1); // On rallume.

    while (true) { // La boucle du jeu :
        image();

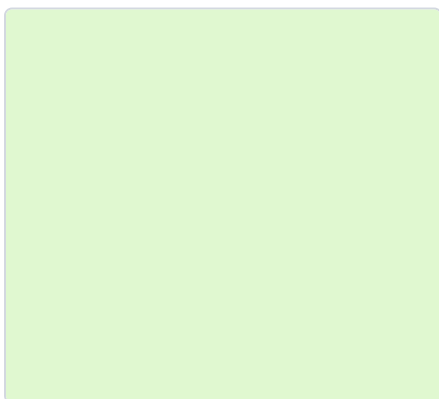
        // 1. La largeur : les deux coins du côté où l'on va.
        // À droite : le coin haut (x + 8, y) et le coin bas (x + 8, y + 7).
        if (bouton(DROITE) && libre(x + 8, y) && libre(x + 8, y + 7)) x++;
        // À gauche : les deux pixels juste à gauche (x - 1) ; x > 0 évite de passer sous 0.
        if (bouton(GAUCHE) && x > 0 && libre(x - 1, y) && libre(x - 1, y + 7)) x--;

        // 2. Puis la hauteur, à part : bloqué d'un côté, on avance de l'autre.
        // En bas : les deux pixels juste dessous (y + 8), à gauche (x) et à droite (x + 7).
        if (bouton(BAS) && libre(x, y + 8) && libre(x + 7, y + 8)) y++;
        // En haut : les deux pixels juste dessus (y - 1) ; pas plus haut que 16.
        if (bouton(HAUT) && y > 16 && libre(x, y - 1) && libre(x + 7, y - 1)) y--;

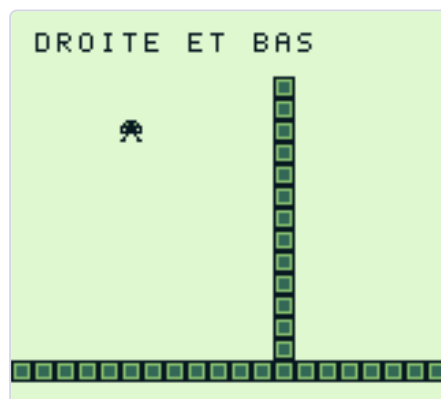
        sprite(0, x, y, HEROS); // On pose le héros.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



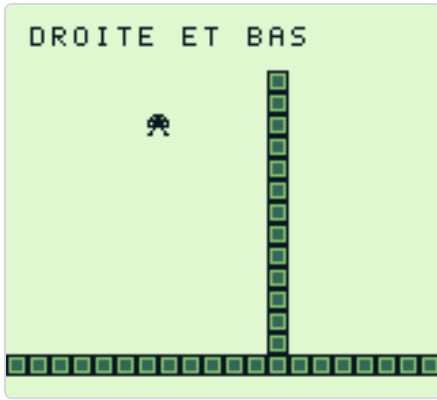
la toute première image — la console vient de démarrer — image 1



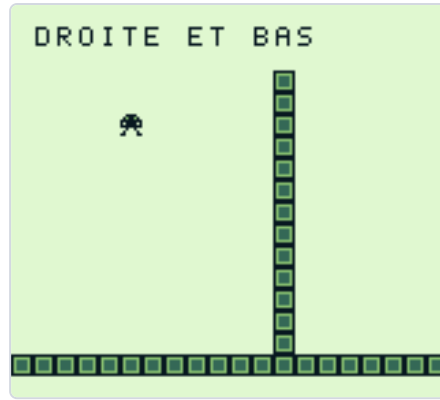
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

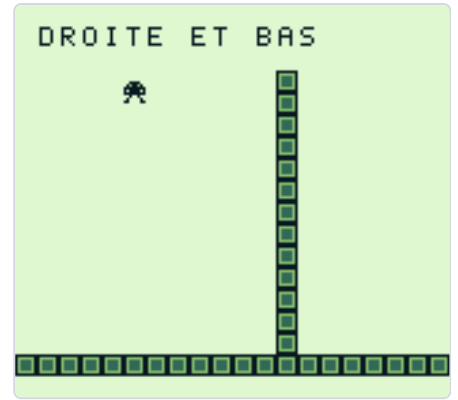
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



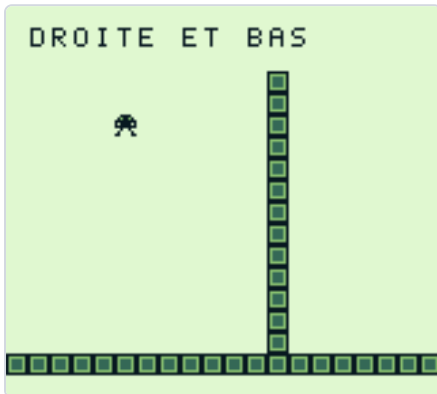
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un mur debout et un sol ; en poussant DROITE et BAS, le héros descend en frôlant le mur, puis s'arrête sur le sol.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le mur arrête le pas à droite : $x = 88$ (88)
- ✓ mais le héros continue de descendre ($y = 100$)
- ✓ il s'arrête sur le sol : $y = 120$ (120)
- ✓ sans entrer dans le mur

← [48. Lire la case devant soi : le mur](#)

[sommaire](#)

[50. Le hasard dans une plage choisie](#) →